

2. Moving from a Monolith to an Event-Driven Architecture

Hugo Filipe Oliveira Rocha¹

(1) Ermesinde, Portugal

This chapter covers:

- How to incrementally adopt an event-driven architecture
- How to decide what functionality to move first
- How to move data from a monolith to a microservice architecture in an event-driven mindset
- Using CDC (change data capture) to move data
- Using events as the source of truth for both the old and new architecture
- How to deal with event-driven microservices that need monolith's data and vice versa
- How to gradually move traffic from the monolith to the new services
- Implementing two-way synchronization and understanding the impacts associated with it

Splitting a monolith is often like an onion. There are legitimate recipes to eat it whole, depending on the dish we are serving. But to release their flavor compounds and achieve their true potential, we have to cut it. Onions, like monoliths, are manageable as a single piece. When we start to cut them down, they become hard to handle. As we cut them, we are never sure how small is small enough, the layers inside the onion start to fall apart and run away from our hand, and depending on the way we cut them, it can rapidly bring tears to our eyes.

In the last chapter, we discussed the advantages and limitations of both monoliths and event-driven architectures. This chapter will tackle how we can start migrating functionality to an event-driven architecture

and discuss several patterns to migrate data using an event-driven approach. We will illustrate how we can achieve this by using an example of an eCommerce platform.

Starting a migration from a monolith can be daunting. Monoliths are often very intricate with complex and tangled logic, often the result of years of unchecked growth, with unsure business requirements and pressing urgency. Modular monoliths are the exception but also rare to find. Even to know where to start the migration and what to migrate first can be hard to decide. On the other hand, event-driven architectures aren't always the best option. Sections [2.1](#) and [2.2](#) will help you decide if an event-driven architecture is the best option and how we can start migrating functionality.

Moving data is one of the parts people most struggle with while migrating to a new architecture. Stateless functionality, like a service that applies a fixed tax to every order without relying on state, is relatively straightforward to migrate. It is greatly simplified by not needing data migrations and keeping data in sync with the old and new architecture. Event-driven approaches provide an alternative to traditional methods by exposing data sustainably and seamlessly. They offer an organic way to keep data up to date in (potentially) real time.

Moving functionality from the monolith to a new architecture usually isn't done (and it shouldn't be done) with a big bang release (transitioning to the new architecture and decoupling the old one instantaneously). Most of the time, we need to deal with the new services requiring data from the monolithic application and the monolithic application still using the new service data. Event-driven architectures also aren't composed of only event-driven services. Often they are composed of both synchronous and asynchronous services. Section [2.6](#) will discuss how we can deal with dependencies between the monolith and new services in both situations.

Even after moving functionality to the new architecture, users and clients don't change all at the same time to the new architecture. We need approaches to gradually move traffic from the monolith to the new services. Furthermore, there is functionality that depends on several different boundaries. We might migrate one of them, but we need to maintain

the interaction between the others. Requesting data from the new services is an option, as discussed in Section [2.6](#), but depending on the use case, it can be hard to do (e.g., if there is considerable logic in stored procedures). Section [2.8](#) illustrates how we can migrate functionality incrementally and maintain two sources of truth.

2.1 Is Migrating to an Event-Driven Architecture Your Best Option?

Chapter [1](#) discussed how a modular monolith might be a good architectural choice and how it can enjoy several properties we often look for in microservice architectures without the challenges of distributed components. This section will teach you to challenge whether an event-driven microservice architecture is the best option and to question if it is possible to solve the issues we face without the added complexity of a distributed architecture.

A survey^{[1](#)} by Kong indicates that most technology leaders believe the failure to adopt a microservice architecture will hurt their company's ability to compete. Microservice architectures have been adopted by many companies as a means to solve scalability and productivity limitations and most of the issues Chapter [1](#) mentioned. Without a doubt, they bear advantages other architectures struggle to provide.

However, we often rush to blindly adopt technologies that work for other companies. By focusing on all the limits the current architecture imposes on us, we often fail to see the simplicity and safety they always provided. Adopting one without careful deliberation often leads to dreadful results. As discussed in Chapter [1](#), the added complexity, the distributed challenges, and asynchronous nature are often mischievous challenges and difficult to tackle with the strategies we often use in a monolithic application.

Take Istio, for example; the community decided to change from a microservice architecture to a monolithic approach gradually. Istio, a service mesh for microservice communication, would apparently benefit from the same architecture that helps others build. However, they soon

found² out the deployment overhead and configuration complexity of several moving parts, the difficulty in debugging, and the overhead of network requests and caches never fully paid off. So they moved back to a monolithic deployment for the Istio control plane, which makes sense for that project.

Segment is also another interesting example; they moved from a monolith to a microservice architecture just to move back to a monolith a couple of years later. They shared an article³ explaining how a small team struggled to cope with the considerable increase in complexity. A single update to a shared library with the old architecture would take a single deployment now would take dozens, one for each service. The developer productivity greatly plummeted due to the large number of services that kept growing. Operational overhead increased steadily with each new service. Eventually, they needed to do something and moved back to a monolith and benefited largely from it.

“Ok, well, obviously you shouldn’t have done things like that” is often a common (and naive) argument when an architecture fails. Instead, we should be asking: “Is a microservice architecture right for that use case?” Sometimes it isn’t; a monolithic approach might be much more adequate. As we mentioned in Chapter [1](#), it is about delivering value to the customer and the best way to achieve it.

The questions we should ask are: “What am I trying to solve? What will I get from it?” And a cutting-edge, cool, brand new, shiny architecture isn’t the answer. Business goals should support the answer and be in line with the actual value delivered to the customer. Using the same architecture or refactoring it in a structured way might yield even better results than a premature microservice adoption.

Let’s illustrate with some examples of answers to that question that might not be a solid reason to advance with an event-driven microservice architecture:

- “We will be able to scale horizontally more efficiently.”: Although that is true, event-driven microservices allow for selectable scalability; as we saw in Chapter [1](#), we can scale a monolith; isn’t that an option?

- “We will be able to develop autonomously.”: Teams working in a modular monolith can also enjoy a high degree of autonomy. The way the company organizes the teams and how autonomously they can work is often a shift in the mindset rather than a technological shift.
- “We will deliver features faster.”: Depending on the size of the team, it can actually hurt productivity moving to a distributed architecture as segment experienced. It is often valuable to understand the root cause of why features are slow. A huge team would indeed benefit from a microservice architecture, while a smaller one might not be able to gain as much and often suffer from other kinds of limitations unrelated to the application’s architecture.
- “It will be simpler to add features for an isolated domain.”: Despite this might be true in most cases, the isolated domain systems are still part of an ecosystem of services, and adding features or new data to entities often lead to the other systems to adapt to the new data or features. Sometimes dependencies between services are hard to manage due to the team roadmap priorities and introducing breaking changes, for instance.
- “We want to adopt a continuous delivery mindset.”: Although a good reason, we should always ponder if we can’t do that with the existing architecture. It is possible to adopt a continuous delivery mindset with a monolith, although it gets more challenging as the teams and application grow.

Here are some examples that often are strong reasons:

- The database can only be scaled vertically and reached a point that it isn’t cost-effective. Monolithic databases tend not to be horizontally scalable. Scaling often reaches a point where a better approach is more effective (especially cost-wise).
- The size of the team. Although the number of teams can be manageable in a modular monolith, it often gets to a point teams outgrow the application.
- Using different languages and technologies or the need to change deprecated ones. Although it is always possible to update an existing monolith, it is often tough and error-prone. Also, it might not be possible to support different technologies in the same application.

We should always question if there aren't other options and if a microservice architecture is the best approach. We should also have strong reasons or concrete limitations to solve. A large scale amplifies most of the advantages we mentioned in Chapter 1, both in usage and developers. If a company didn't reach that stage, it will likely struggle with the complexity more than enjoy the benefits.

As we mentioned in Chapter 1, not having clear domains and boundaries is also a strong reason to postpone that migration. In the early life of a company, domains are prone to change. Changing the domains and boundaries in an event-driven microservice architecture is costly and laborious. Having a monolith (even a bad one) at least guarantees there is a physical place to validate the existing domains and refactor them with less effort.

However, often teams tend to outgrow a single application. Managing changes and releases becomes harder as the teams and application grow. Guaranteeing the isolation properties of modular monoliths becomes increasingly harder. Event-driven microservices provide a good alternative to solve these problems, provided we have a clear objective.

2.2 Moving to an Event-Driven Architecture, How to Decide Where to Start

This section will discuss how we can choose the most appropriate functionality to migrate from a monolith to an event-driven architecture. We will illustrate an example of an eCommerce platform monolith and explain how we can choose one of its modules to create a new service.

Most complex applications have several domains, and they interact with each other to achieve a higher-level process. A bounded context is focused on a piece of the overall functionality and contains a given domain model. For example, an eCommerce platform might have several bounded contexts, being one of them the order management. The order management bounded context could have a core domain model that conceptually illustrates orders (bounded context is a domain-driven design term that we detail in Chapter 3; for now, see it as a domain boundary).

In a perfect world, we would just stop product development for a few months while re-architecting the whole system. This situation is rarely the case since the business must still operate and evolve as the new architecture is being built. An important consideration is to migrate functionality incrementally; bear in mind that often we need to have both old and new systems running simultaneously. It is imperative to build a sustainable way to gradually migrate functionalities to the new architecture to avoid big bang releases. Migrating gradually also provides a way to deal with the event-driven architectures' challenges on a lower scale. It also is easier to guarantee a safe way to roll back the changes if needed. We will get some parts wrong; it is vital to guarantee the business's reliability and give us the space to learn from our mistakes. Event-driven and the patterns we describe in this chapter are built in this mindset to allow incremental delivery of functionality.

The first step to start moving functionality to an event-driven architecture is understanding the existing domains and bounded contexts. Often monoliths feel like the example illustrated in Figure [2-1](#).

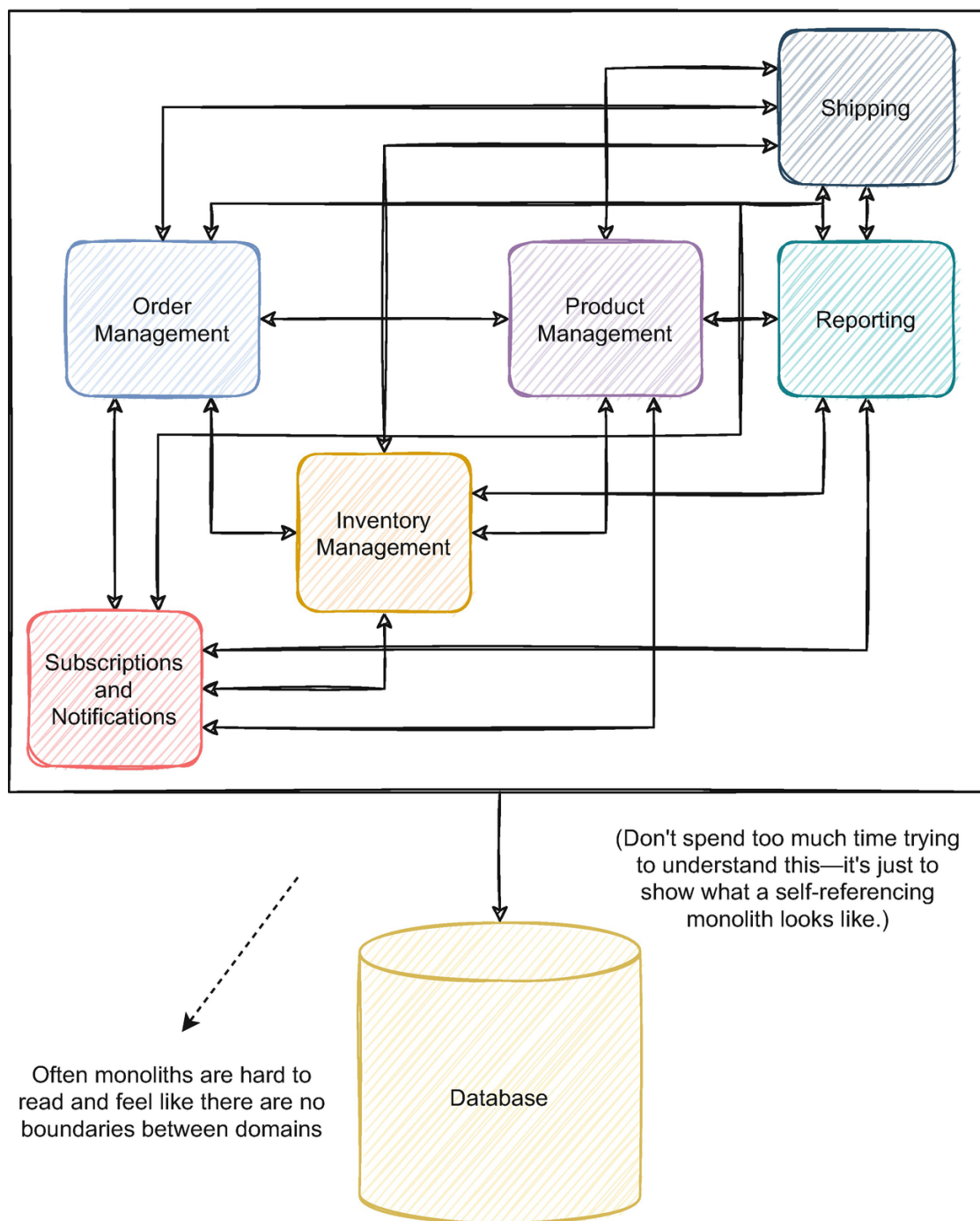


Figure 2-1 Example of how we often feel about monoliths. Illustrates an example of an eCommerce platform where everything references everything

It might be overwhelming to analyze all the bounded contexts in massive patchwork monoliths. The first few tries might feel like everything references everything, and there is no clear distinction. DDD (domain-driven design) has techniques that can help with this, further detailed in Chapter 3. However, having an existing application that already describes these relations can help in understanding how a business operates, hence the importance of sometimes starting with a monolith and then moving incrementally to an event-driven microservice architecture.

If you are dealing with a patchwork monolith, it might be useful to refactor it to a modular monolith. By doing so, we might even solve the initial reason that made us consider moving to an event-driven architecture. It will also underline the existing bounded contexts and make their dependencies visible. If we can't build a well-structured monolith, why do we think we can create a well-structured event-driven architecture? With that information, we can make an informed choice on which module to migrate first. Do not underestimate this effort as domain knowledge is essential to build the new architecture.

Even if we don't choose to change the existing monolith, trying to understand how we could change it to a modular monolith and the underlying exercise of comprehending the boundaries and bounded contexts is pivotal to understanding which domain to move first. Eventually, we will reach a design similar to pictured in Figure [2-2](#).

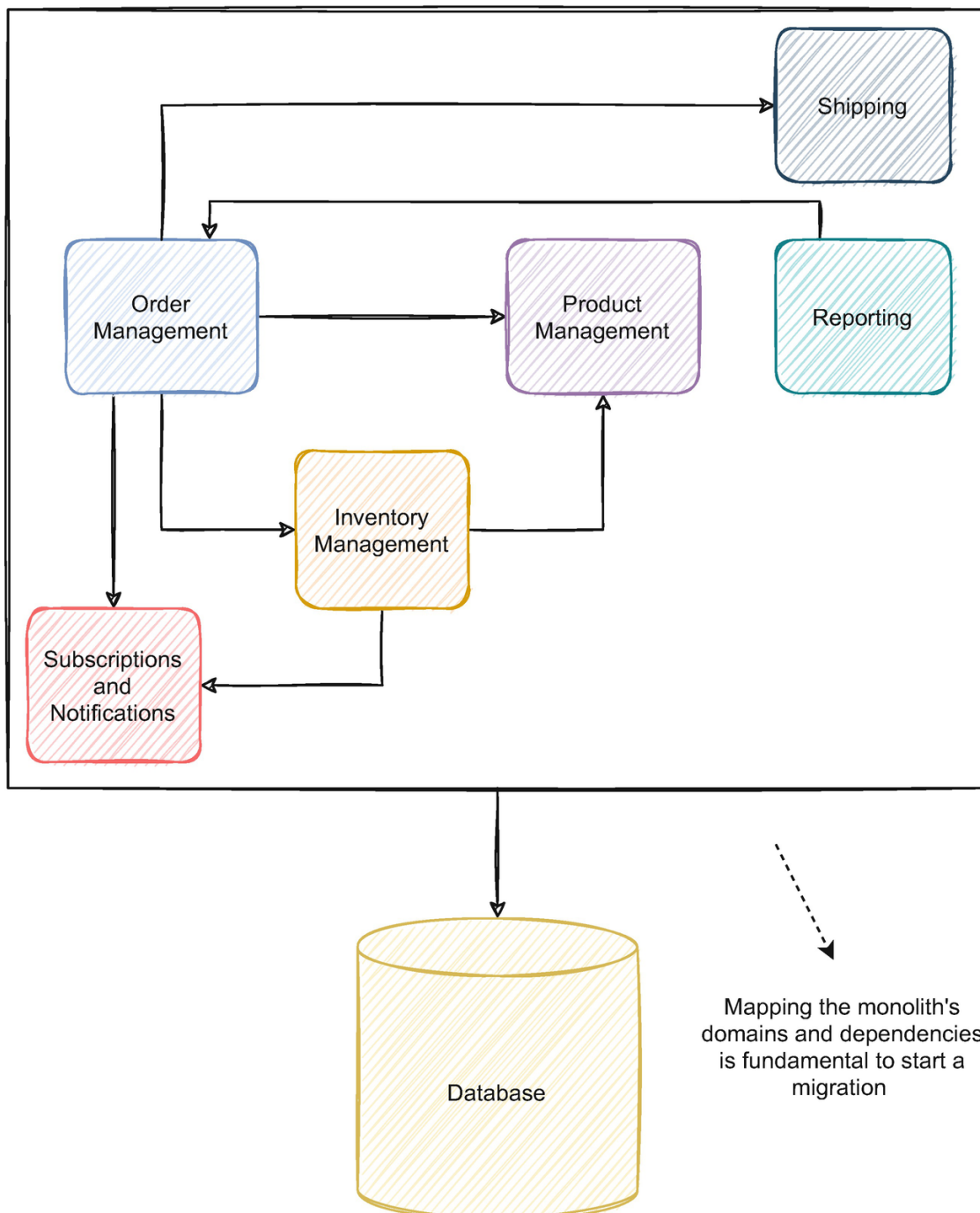


Figure 2-2 Example of an eCommerce platform with several interactions between bounded contexts

Once the business features and how they map with each bounded context are clear, we can design the domain model. These steps are fundamental to start the migration. Those bounded contexts will eventually map to different services responsible for managing the operations in each domain's scope.

We have our bounded contexts and dependencies mapped (as detailed in Chapter 3); now what? We need to decide which module to migrate first. Often it is a weight between the difficulty in moving a module, how that module would benefit from the move, and how lenient the consis-

cy guarantees can be. As we discussed in Chapter [1](#), event-driven architectures feature eventual consistency; a functionality might allow smaller or larger inconsistency windows depending on the impact of that weak consistency model. It is wise to start migrating features that allow lower consistency constraints to give us space to tune the system to guarantee the inconsistency is small enough that won't impact the system's users (how to tackle eventual consistency is discussed in Chapter [5](#)).

In the example in Figure [2-2](#), the order management module might benefit significantly from an event-driven architecture to ease the process of scaling in the face of load peaks. However, it might not be easy to remove the module because it depends on the other four modules. The shipping or reporting module could be a good alternative since each one only has one dependency. Shipping might need stronger consistency guarantees than reporting, which typically is asynchronous; often reports don't need to be accurate to the last second. Following this line of reasoning, reporting could be a good option to migrate first.

We should try to find functionality that is easy to migrate, that largely benefits from the migration, and it is not critical to have strong consistency guarantees. All modules are liable to be relocated, but doing an easy part first can be an excellent way to get feedback on how the migration is going and pave the way for the system's more challenging pieces. It also guarantees you face the challenges of distributed architectures gradually and can sustainably respond to them.

It is also important to understand how the migration to the new architecture is going. We are doing this because we are trying to deliver value to the customer or solve an issue in the current architecture. Our gut feeling is important, but as engineers, we try to drive our decisions with data. What should we measure? It is often related to the answer we gave to the question in Section [2.1](#) on the reason we are migrating.

If we are trying to improve the team's autonomy and productivity, it might be interesting to measure the dependencies between teams, feature branch longevity, or merge requests. Long-lived branches are prone to get stale due to other teams conflicting changes in the master branch. It is often the source of complicated merges and a high number of conflicts; it is

one of the typical challenges teams have when they outgrow one application.

It can also be useful to measure the deployment process. If we are trying to shift to a continuous delivery mindset, the number of deployments, cycle time, and lead time might be interesting metrics. They also relate to the number of dependencies between teams since with high coupled code typically found on monoliths, features sit on ready-to-merge state longer. The shift should improve the overall time a feature takes to be deployed.

If we are trying to improve our scalability capabilities, the number of incidents due to load peaks or measuring response time might be useful. The tier of the current machines for both the application and database might also give some insight into the capacity the application has to support a growing business. As we discussed, often monoliths struggle with load and when the only way to scale is vertically (by increasing the machine's resources) it can constrain the business growth. As we migrate to an event-driven architecture, gradually the load will be distributed between the new services, so the high-tier machines shouldn't be needed anymore.

We also can unconsciously overwork metrics, and with enough creativity, they can reveal anything. To some extent, we should base our decisions on metrics, but it is as important to know the feeling of the people working with the system. Understanding their problems, concerns, and opinions on the state of the migration can give an overall view of how it is going. Be sure to check with the team regularly to understand their experience; their feedback is the most valuable metric you can find.

2.3 Using an Event-Driven Approach to Move Data from a Monolith

The last sections discussed the decision to move to an event-driven architecture and how we can decide which module to move first. This section will approach how we can expose and transfer data from the monolithic application to the new architecture.

Using the example in Figure 2-3, let's say we decided to migrate the reporting module to a new service. Remember how we discussed why it is essential to do an incremental migration? We could start by building the reporting service outside the monolith.

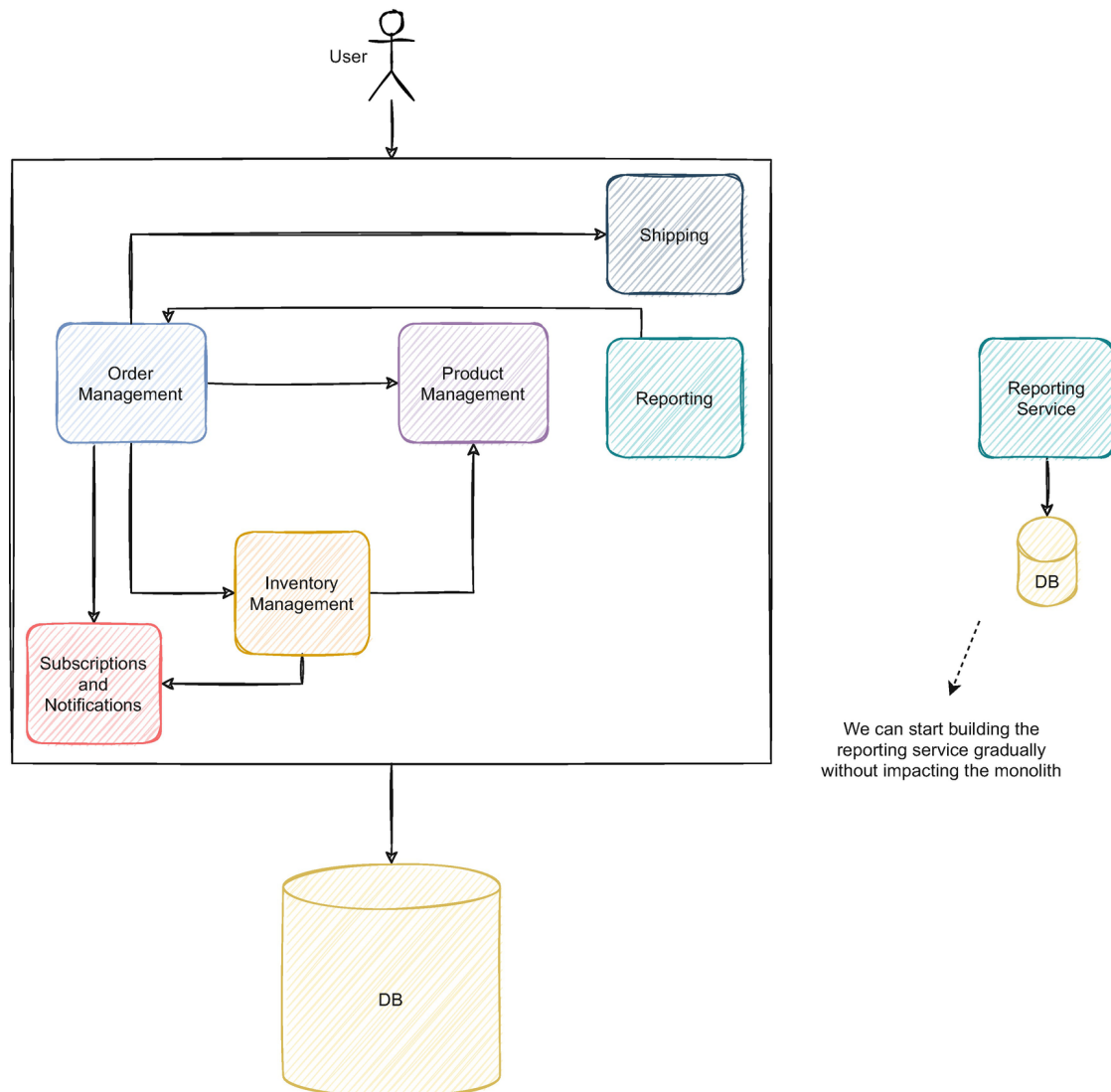


Figure 2-3 First step in migrating the reporting service to an event-driven architecture. It is important to do the steps incrementally by building the reporting service independently from the monolith

By building the reporting service without any interaction with the monolith, we can avoid any impact on the other teams working on it. We can also incrementally add functionality to the reporting service. We could first deploy the service in the live environment without any functionality and then gradually add feature by feature until the service has all the monolith's reporting functionality.

A common question is whether to copy the code as is in the monolith or to refactor it. It depends on the monolith; on a well-structured and organized monolith, it might make sense to copy the code as is; it will save

us time. However, most often than not, the monolith's model might not be up to date to the incoming requirements. The migration is a good opportunity to refactor it and build it more cleanly if needed. We will soon add an event queue between the reporting service and the monolith so they can have different models and behaviors as long as the monolith obeys the event's contracts.

Just bear in mind that while we are migrating, both services have the functionality. A change in the monolith's reporting module due to bug fixing or new features means we have to change the reporting service accordingly until we descope the functionality in the monolith.

Let's say the reporting service generates and saves a report for each fulfilled order. We already migrated the functionality to create the report. But we need the order information to trigger the report creation and to fill the report's data. As we see in Figure [2-3](#), the monolith's reporting module has a dependency on the order management module due to fetching information from that module. We need to somehow expose the order information to be consumed by the reporting service. The next step is to step up a queue between the service and the monolith; the reporting service will use that queue to react to events and generate the reports, illustrated in Figure [2-4](#).

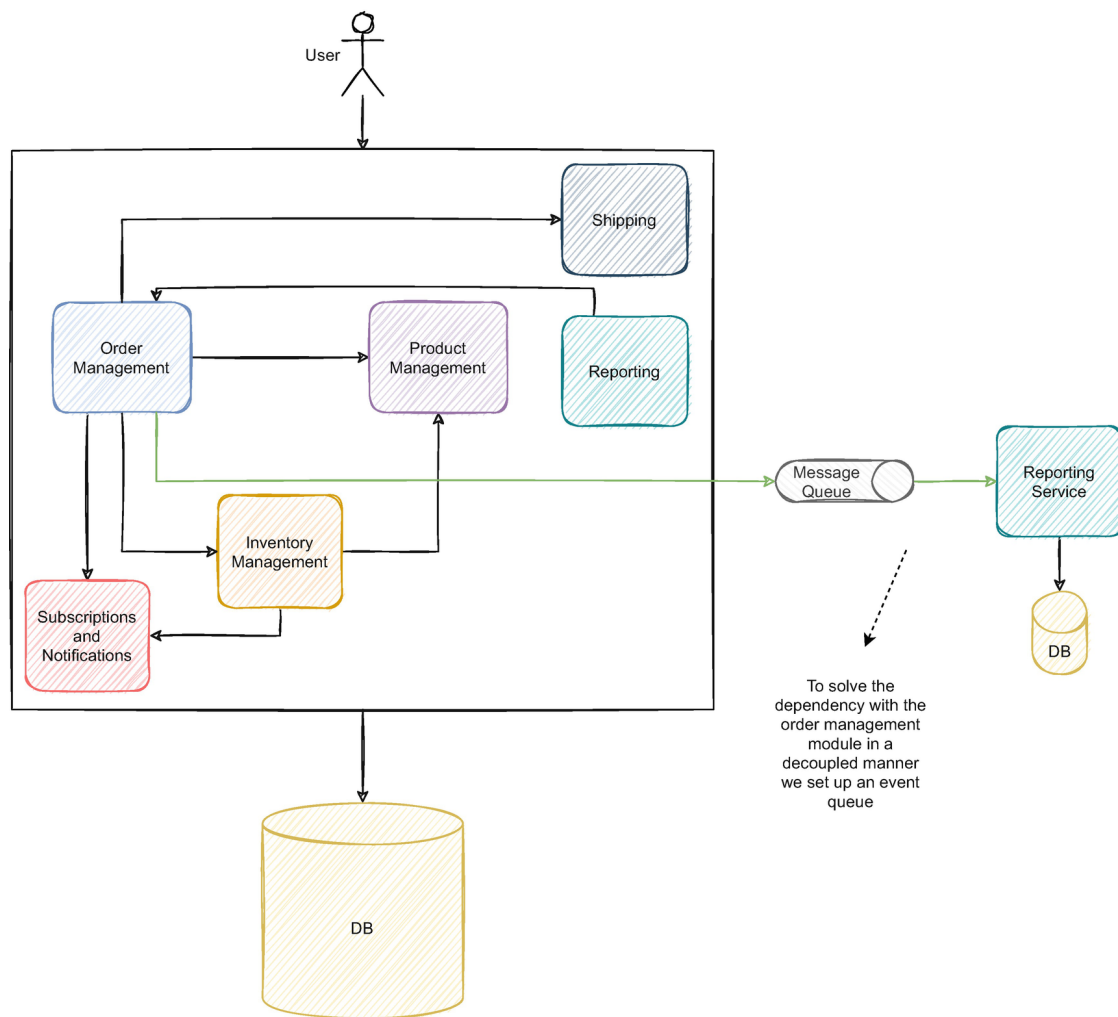


Figure 2-4 Second step in migrating the reporting service to an event-driven architecture. Set up a queue with the order management information to be consumed initially by the reporting service

An essential step in building the reporting service is defining the contracts that the service will listen to. Event schema usually triggers a lively debate and is a core design that affects the downstream services. It is crucial to design them right; otherwise, they can trigger difficult challenges to the services that consume the event. How to design the event schema is further discussed in Chapter 8. At this point, let's worry about creating a contract that fulfills the needs of the reporting service.

Once we set up the order management event publishing and the reporting service event handling, we are already able to expose functionality in the reporting service. For recent orders, users can already use the reporting service to obtain the reports. But what about all the reports that were created until now? We just migrated functionality, not data.

There are several strategies to migrate data from existing databases. You probably had to do this one time or another and likely will know how

hard it is. Section [2.9](#) approaches some of these alternatives. Without events, we could run a batch from time to time to synchronize the monolithic database to the new service, introducing higher complexity with higher delay. We could temporarily access both databases with all the issues that can cause.

The beauty of event-driven services is that they provide a way to seamlessly expose that information without these complicated ad hoc processes. Events are the source of truth, and we can use them to transfer data to any service that needs it. We can leverage the fact we already handle events and use the same strategy to migrate all the existing data that we need along with the data that is happening now.

In the example of Figure [2-4](#), we could trigger a job in the order management service to publish all the order information to the event queue. When the reporting service starts to consume, it will consume all orders and generate all reports that it needs. It would also be a great way to validate the service's correctness; the generated reports would have to be the same as those existing in the monolith.

Using an event broker that keeps the events and doesn't delete them upon consumption also opens the way for exciting possibilities. Imagine we introduce a bug in the reporting service, and we need to generate the reports again. We could simply start reading the queue from the point where the bug was introduced. If there was a feature to change the reports to have different information, we could simply start reading from the queue's beginning and generate the new reports. Operations that otherwise would be cumbersome to do, with error-prone manual interventions, are suddenly built in and can occur organically.

But we are not done; we migrated the functionality and the data, but the user is still using the monolith to obtain the reports. Now we can inform the consuming applications to change to consume the reporting service. We can also have a proxy on the layer above the monolith redirecting the calls to the new service. Section [2.8](#) further discusses these options.

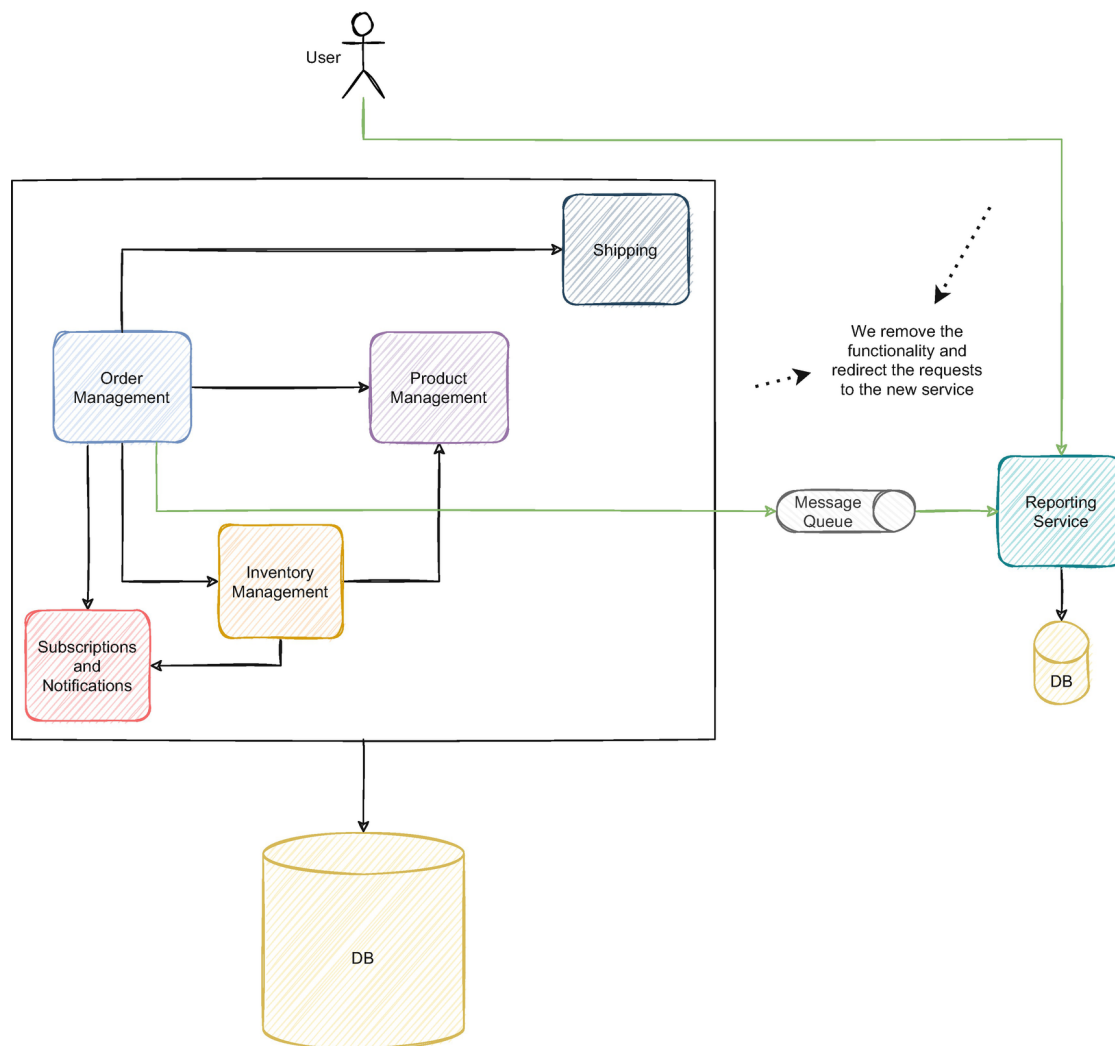


Figure 2-5 Third step in migrating the reporting service to an event-driven architecture. We finalize the migration of the reporting functionality by removing the old module

Once there are no calls to the monolith’s reporting module, we should remove it from the monolith, as depicted in Figure 2-5. If we incrementally do this to each module, we gradually decompose the monolith into smaller services.

We also built the foundation for other services to build on top. If another module needs the order management information, it’s already exposed. We can start building the service and plug it into the existing message queue. Even the order management module itself, when we create the new service, can migrate the data by merely streaming the events in the queue by reading it from the beginning. Once we migrate the order management module, the reporting service would consume events from the new service rather than the monolith. The decoupled nature of event-driven services allows the order management service’s migration without changing the reporting service. In a traditional microservice architecture

with synchronous requests, this change would need coordination to at least shift endpoints from the monolith to the new service.

This way, an event-driven approach enables data streaming from the monolith to the new services. It avoids other complex alternatives to split or share the database without a custom batching process or other manual options. Streaming the data serves both as a way to share real-time information to the new services and as the history of data, easing the migration process. It is also a very intuitive, scalable, and organic way to distribute data throughout the architecture.

Adopting an event-driven service has its drawbacks; as we mentioned in Chapter [1](#), the reporting information is now asynchronous and might not enjoy the strong consistency guarantees it enjoyed while inside the monolith. In this particular case, reports are typically asynchronous, so it seems a good fit. However, other functionalities might have stronger consistency guarantees, for example, guaranteeing the consistency with several tables by using transactions. Achieving consistency is still possible through process managers and Sagas, as detailed in Chapter [7](#). How to deal with eventual consistency is discussed in Chapter [5](#).

2.4 Using Change Data Capture (CDC) to Move Data from a Monolith

The previous section discussed how we could share the monolith's data by publishing events in internal modules. However, capturing every functionality that might change information can be challenging. Monoliths also tend to have a considerable amount of logic in the database's stored procedures, making it even harder to publish changes to an event broker. This section will discuss how we can use a similar event-driven approach but using CDC (change data capture) to extract the data.

One of the situations where I experienced firsthand the difficulties of extracting monolith's data was when the team was trying to migrate the inventory management module from the backoffice application (a large patchwork monolith) of an eCommerce platform. The first approach we tried was to call the new service every time any functionality changed in-

ventory data. By making two requests, one to the new service and maintaining the monolith calls, we were able to verify if both databases held the same data. After changing a few missing flows, it looked great; the regressions we did on every functionality were working fine; both databases were the same in every quality environment.

When we reached production, however, it was a completely different story. The new service data somehow was slowly diverting from the monolith (the current source of truth). Although the high majority of flows were calling the new service, we found out there were still some stored procedures (with some hundreds of lines each) that changed the data. The support teams, from time to time, also did some data fixing directly on the database. There were also mushroom applications outside the monolith that changed the data. Mushroom applications are ad hoc applications that often pop up throughout the company, created outside the standard architecture to achieve a small objective. It can even be an excel sheet accessing the database. Many companies have several of these created by operational teams to help with ad hoc operations like data analysis or automation of a common task. Due to these reasons and others, we soon found out that completely covering all flows that accessed that data would take a substantial effort. There are times where it is overly complex to change an existing monolith to follow, for example, the approach in Section [2.3](#). Sometimes it's not even possible when using third-party vendor monoliths.

CDC is a pattern to identify and capture individual changes made to a database or application. Using CDC can be a valid alternative in these situations since we can capture every change directly on the database. There is also built-in functionality on many databases that support CDC and provide easily accessible ways to extract data.

Instead of natively extracting and publishing data, there are also several frameworks to extend this functionality; for example, Kafka Connect uses CDC and connectors to the database to extract data and load it to Kafka. Apache NiFi is also a reliable option that provides the same functionality. These frameworks are easy to bootstrap and offer an easy way to ex-

tract data from the database, mostly through configuration. It is also possible to create our custom transformations.

However, these frameworks have a few drawbacks:

- If we are trying to track down a live incident, it's hard⁴ to locally debug the connectors. Building unit and integration tests can be challenging due to the references to all the internal Kafka classes.
- The level of customization might not be the most adequate for our requirements and might have limitations. If we need to enrich the data with an external source on the connector, it becomes hard to extend (although we should keep that logic out of the connector, it isn't always easy).
- These frameworks tend to expose the underlying database data models adding additional coupling. We might prefer a custom application in charge of extracting, creating, and managing its events. Often it is preferable to endow the events with domain meaning. An update of a few fields might have more business value than the technical definition of a few columns that were updated. Maybe updating the stock column means making a reservation business-wise. The events themselves should reflect that business intent. That can become hard to translate using these frameworks.

Depending on your requirements, building a custom solution to analyze the database CDC can be a preferable option and give you higher flexibility. On the other hand, using a framework can significantly reduce the implementation overhead and set up a solution with little effort.

In the same example in Figure [2-2](#), how could we migrate a module using CDC? The first step would be to set up a service or a connector to the database, depicted in Figure [2-6](#).

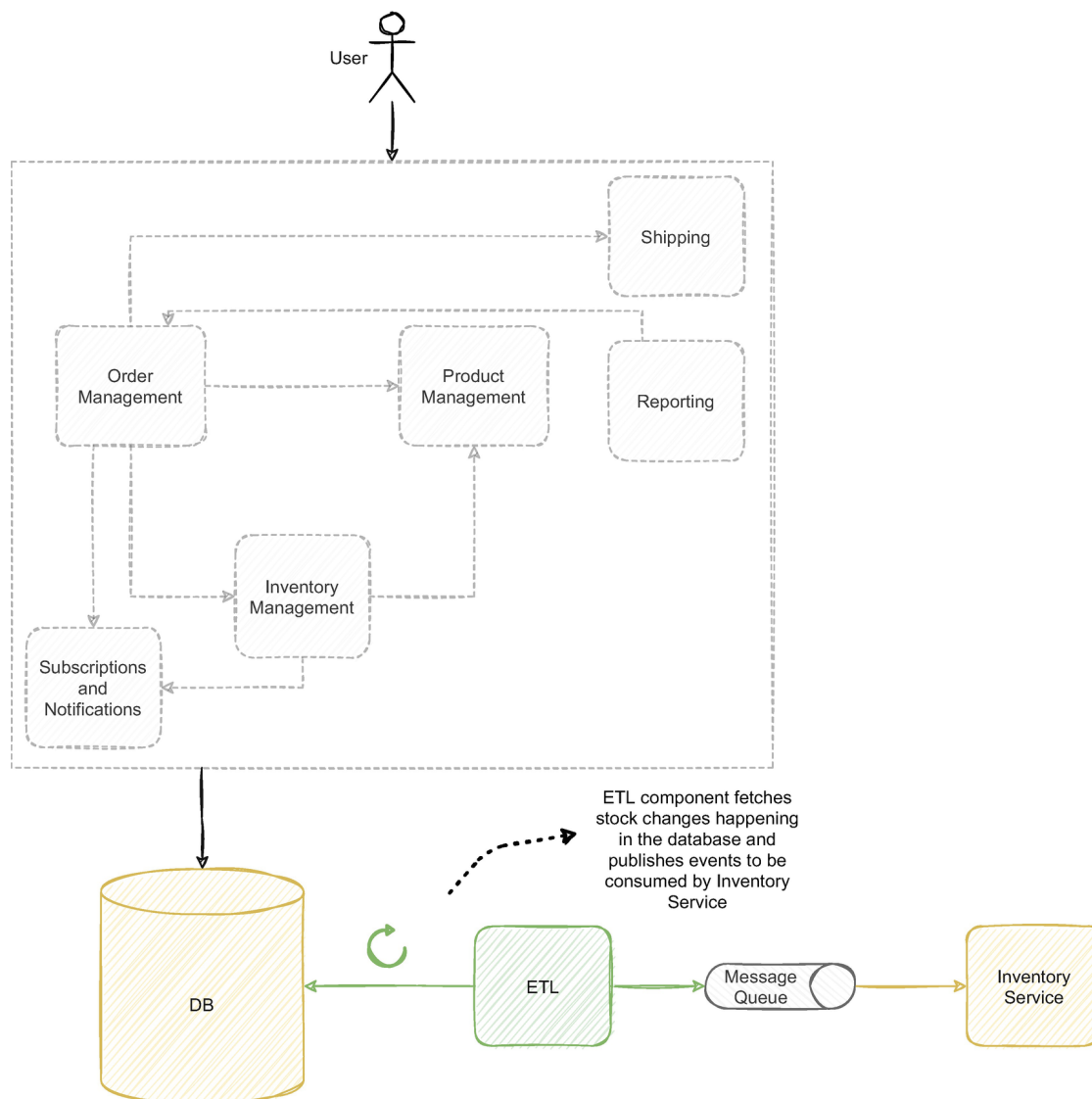


Figure 2-6 Using CDC to migrate data from a monolith’s module. We set up a component to read the databases’ changes with CDC and publish them to a message queue

The ETL (extract, transform, and load, although the load is to a message queue) component processes the changes, converts them to events, and publishes them to a message queue. In the same example with the reporting service, the component would poll for changes in the databases’ CDC mechanism in the order tables and publish them to the message queue. It also provides the role of decoupling and hiding the undesired data CDC provides to external events. If we use the frameworks mentioned before, they might expose more information than needed and leak information about the underlying data models. A custom component would abstract these concerns and publish events that made sense to the new services.

The ETL would probably need to bootstrap the current information by fetching the current state and publishing it to the message queue to boot-

strap the new service. To descope the reporting module on the monolith, we could then build the reporting service and listen to the events in the queue. After that, we would change the consumers to the new service and descope the current functionality. Our final mission is to descope the ETL component and have the new service be the source of data.

An important consideration is as the new architecture grows, we should continuously add functionality to the same ETL component or add different individual components. Well, it depends on the use case, but here are some guidelines to help you decide. If there is only a small team using the ETL component, then one component is probably preferable; using a framework such as Kafka Connect could also be a good option. When there are several teams with diverse bounded contexts, building an ETL per bounded context might be advisable. The implementation overhead will be larger, but it will give each team independence to create the events for their context without dependencies between other teams and models. It will also encourage them to understand how the data is organized on the monolith, what events make sense, and how they interact with the new services.

2.4.1 Event-Driven and Change Data Capture (CDC), a Real-World Example

Custom querying is one way to implement CDC. We can extract the data by querying the database, with or without filters. We can also bulk load all data to the message queue, sinking all existing information to an event stream. After the bootstrap of all existing data, the process would load changes after a given watermark, which can be a timestamp or an incremental id or version. We would publish all changes that occurred after the watermark to the message queue.

It is also possible to customize the query to filter only a set of records. For example, if the table has a type, it might be relevant only to publish the records of a given type or segregate types into different queues.

Any database supports querying, so it is always an option even if the database technology doesn't have a built-in CDC process. It is also very flexible since we can adjust the query to the way we want the data. On

the other hand, the more complex the query is, the higher the impact on the database's resources.

It also relies on a last updated column, which might skip multiple updates to the same record. Most of the time, this isn't an issue, but if we want to capture the user's intent and how the data evolves through time, it won't be as accurate since it might merge several updates. Squashing several updates might be an issue depending on the way the old and new architecture is designed. For example, if we have a table with generic properties that each has a type, we might have decided to separate each property in independent models on the new architecture. If so, an update to the type would mean the deletion of the old resource and creating a different one. Having information on only the latest state might not be enough to apply the same state to the new architecture.

Deletions are also hard to detect for the same reason unless we implement soft deletes (by having a state column). Soft deletes consume more resources since the record isn't removed. Besides that, it surely wouldn't be unprecedented if someone hard deleted a soft delete table (by actually deleting instead of updating the column).

Many database technologies have an internal operation log that records every change operation that occurs in the database. That log is also used to restore the database to a correct state if a catastrophic failure occurs or to replicate information to other instances. It is a fascinating mechanism very similar to event sourcing and event-driven principles. Many of them also provide CDC functionality by using the operation log to expose the changes that occurred in the database.

Using the CDC functionality, we can fetch every change operation that occurred in the relevant tables. Unlike custom querying, even if a record is updated multiple times, it will provide information for each update. It also records deletes, allowing for a more realistic view of the data evolution without soft deletes. Performance and resource-wise log scanners have less impact than querying since it relies on the operation log rather than querying specific tables.

However, they are highly dependent on how the database technology exposes this information. They might not have all the information needed or rely on internal identifiers that are hard to track. Most technologies that expose this functionality have detailed information on each change, which often implies they expose the underlying data model, raising the need for an adaptation and encapsulation of only the relevant information.

CDC Example Using SQL Server and Kafka

Let's illustrate an example of moving data using CDC with SQL Server. Using the same example monolith of Figure 2-6 and assuming we want to move the stock data from the monolithic database to a new service, we could build a topology similar to Figure 2-7.

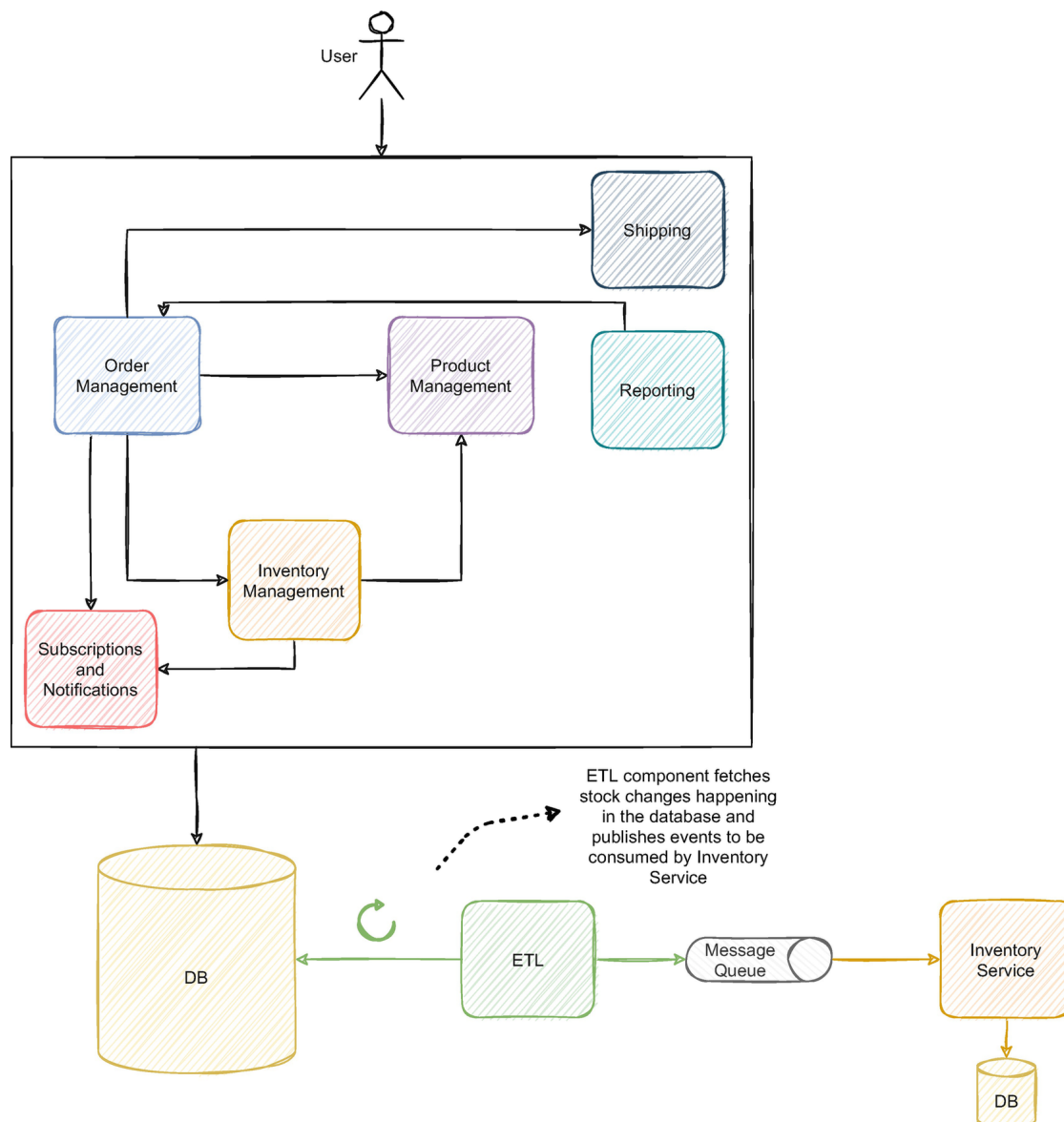


Figure 2-7 Example of using CDC to move data from a monolithic database to a new service (inventory service)

We can access SQL Server CDC information through CDC specific tables that the database creates when enabled. SQL Server has a transaction log with every change that occurs on the database. This log is essential to guarantee that the database is in a consistent state. CDC uses this log to understand what operations are happening in the database and populates the CDC tables with this information and some metadata. We can use the data populated in those tables to understand the changes that occurred in the database.

Let's say we have a Stock table that has the stock for each size of each product. Listing [2-1](#) shows that table and how to enable CDC for the database and that table.

```
1  -- Create table stock
2  CREATE TABLE Stock #A
3  (
4      ProductId int,
5      Size varchar(10),
6      StoreId int,
7      Quantity int,
5      primary key (ProductId, Size, StoreId)
4  )
7
8  -- Enable CDC at the database level
9  EXEC sys.sp_cdc_enable_db #B
10
11 -- Enable CDC at the table level
12 EXEC sys.sp_cdc_enable_table @source_schema = 'dbo',
@source_name = 'Stock', @role_name = NULL,
@supports_net_changes = 1 #B
#A Create stock table definition
#B Enabling CDC at the database and table level
```

Listing 2-1 Create Stock table and enable CDC

Every time a change occurs on the Stock table, the agent records the change in the SQL Server transaction log (even without CDC enabled). The transaction log reader agent will read the changelogs from the transaction log and insert those changes in the CDC tables. The CDC table was created once we ran the command on line 12 in Listing [2-1](#), and we can fetch the changes happening in the database by querying that table.

```
1  -- Some changes to stock records
2  insert into Stock values(153854, 'XS', 914, 1) #A
3  update Stock set Quantity = 2 where ProductId = 153854
and Size = 'XS' #A
4  insert into Stock values(153854, 'S', 914, 1) #A
```



```

5    delete from Stock where ProductId = 153854 and Size =
'S' #A
6
7    -- Fetch changes
8    SELECT * FROM [cdc].[dbo_Stock_CT] where __$start_lsn >
0x00001058000007110003 #B

```

#A Some changes done in the Stock table

#B Retrieving the CDC logs in the table dbo_Stock_CT (the value will change from your use case)

Listing 2-2 Changes to stock records and retrieval of CDC logs

Listing [2-2](#) shows some changes done to the Stock table. We inserted two rows and then updated one and deleted another. Querying the CDC table retrieves the result illustrated in Figure [2-8](#).

#A		#B	#C	#D				
__\$start_lsn	__\$end_lsn	__\$seqval	__\$operation	ProductId	Size	StoreId	Quantity	__\$com...
0x000010...	NULL	0x0000...	2	153854	XS	914	1	1
0x000010...	NULL	0x0000...	3	153854	XS	914	1	1
0x000010...	NULL	0x0000...	4	153854	XS	914	2	1
0x000010...	NULL	0x0000...	2	153854	S	914	1	1
0x000010...	NULL	0x0000...	1	153854	S	914	1	1

Figure 2-8 CDC table results from the changes in Listing [2-2](#)

The CDC tables have some columns directly related to SQL Server but might be relevant to some use cases:

- __\$start_lsn and __\$end_lsn (#A) are the log sequence number (LSN) assigned by the SQL Service.
- __\$seqval (#B) gives information about the order of the operation related to other operations in the same transaction.
- __\$operation (#C) indicates what was the operation performed on that row (1 is delete, 2 is insert, 3 is the row information before the update, 4 is the row information after the update).
- All the stock table columns (#D) with the data when the change occurred.

The query in line 8 of Listing [2-2](#) would be in the ETL component. This component polls the database to fetch the records that happened after the last LSN that it processed (the SQL Server function fn_cdc_get_al-
l_changes_dbo_stock could also be used to fetch the changes on a specific

interval). The query would return the changes to the table between the last polling iteration and the current time. An important consideration is to fetch only a max amount of changes (like 1000 rows each time) due to performance reasons and resource optimization.

Once we load the changes, we would map the data fetched from the database to the stock events. For example, operation 2 (insert operation) could generate a StockCreated event, and operation 4 (update operation) could generate a StockChanged event. Once we map the data from the database to the events, we could publish them to Kafka. The inventory service would handle those events and update its internal state with those events.

The CDC table gives a lot of information about each change and how the data was before the change. Update commands provide details on how the data was before and after the change. Some use cases might need this kind of detail if we don't need only the latest state but also need the state before the change. For example, we could move stock across two products if we updated the product id in the Stock table. Depending on how we design the system, it could be necessary to remove stock from one product and add stock to the other. Reflecting the change on both products might need two different events with information before and after the update.

CDC has the disadvantage of recording a large amount of information about each change. This can lead to excessive resource consumption in high usage tables and databases. We might need to implement an aggressive retention policy in those cases. In the case of SQL Server, there is also another option using Change Track, which is relatively similar to CDC but compares changes between two versions. It doesn't give the detailed data evolution CDC does, but when we need only the latest state is fairly lighter than using CDC.

2.5 Migrating Data from a Monolith: Event-Driven As a Source of Truth for Both Systems

The last sections detailed how we can extract data through events from a monolith. This section will discuss how we can use events to be the single source of truth for both the monolith and the new services.

A pivotal mindset to embody while adopting an event-driven architecture is to embrace events as the source of truth. Until now, we discussed how we could shift the source of truth from the monolith to the new services. However, as long as the information relies on the monolith, it will be the source of truth. Shifting consumers to the new applications might not be an activity we can do autonomously; some of them might be external consumers that might need to adapt to the new services. If so, the shifting action can take a considerable amount of time, during which we are stuck with the monolith being the source of truth or having simultaneously two sources, as we will discuss at the end of this chapter.

An interesting approach is to have the monolith react to a queue. Adding a queue can be harder or easier depending on the monolith. Some monoliths already react to message queues, so plugging a new service into the same queue and routing the messages accordingly to the new and old architecture would be a good approach. If the monolith doesn't have a queue, we could transform the requests into messages. The transformation could be done by the external consumers, an intermediate component, or in the monolith itself.

Although similar, it differs substantially from the approach described in Section [2.3](#); instead of the event publishing being a new secondary operation from the initial flow, it becomes the trigger to the existing process. This way, the message becomes the source of truth for both the old and new flow.

This alternative is illustrated in Figure [2-9](#) using the same example as before; a message queue is the input for both the monolith and reporting service. Having a queue to trigger the process promotes the mindset shift to prioritize event design since the message is pivotal for both systems to operate.

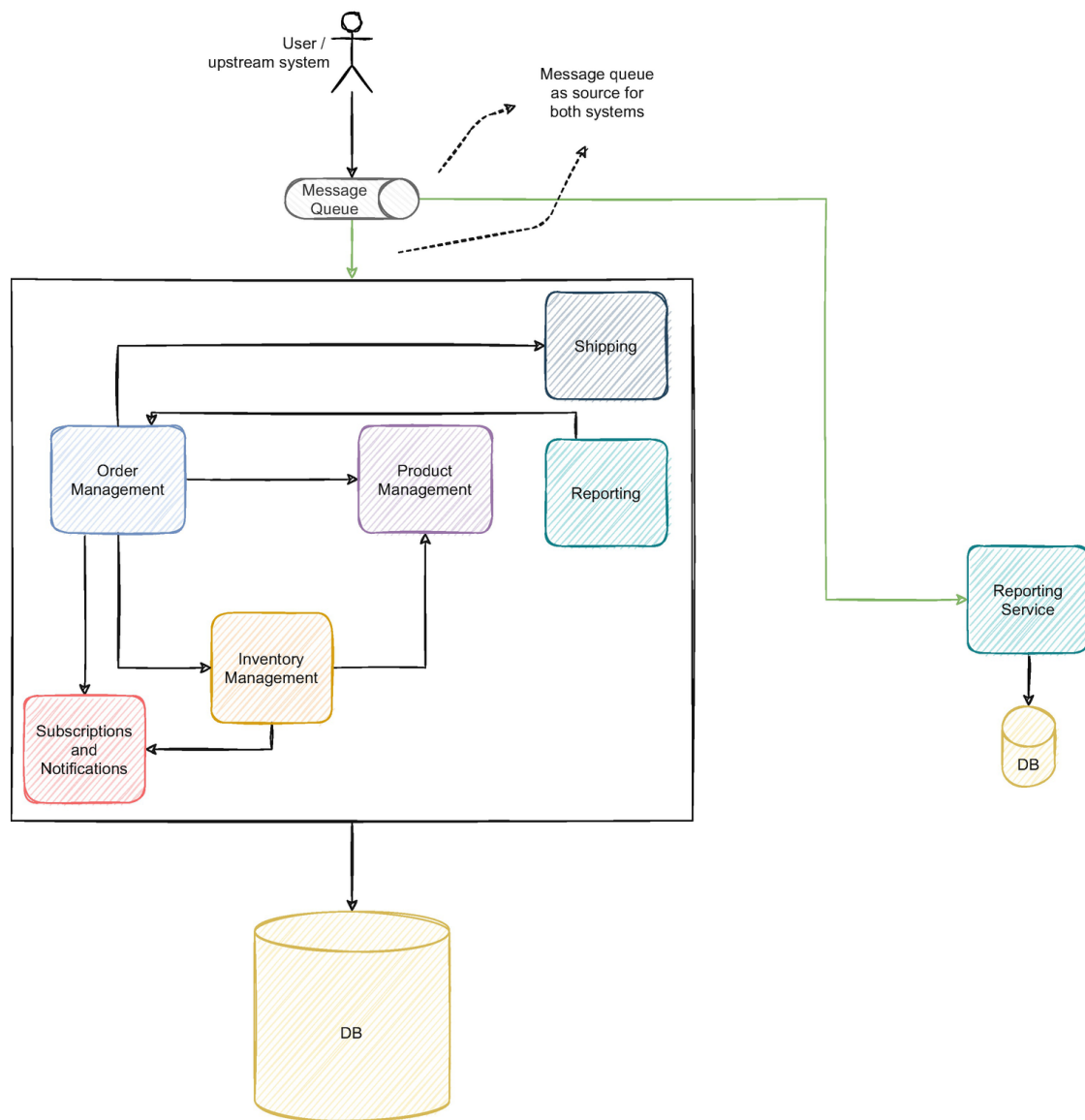


Figure 2-9 A message queue is the input for both the new architecture and the monolith, being the only source of truth

Once we get the queue set up, we could gradually change the message consumption from one system to another. We could roll out based on the messages' content; if we are publishing order information, then we could roll out by country, for example, starting with all orders from "CA" instead of all countries at once. Once the reporting service is consuming all countries, we could remove the monolith's reporting module.

If it's not possible to change the upstream systems to publish to a queue or add an intermediate component, we could split the monolith flow. The first step would be to receive the request, send a message, and then continue with the rest of the flow by handling the message. The monolith itself has only limited benefits to having a queue but would set the new system's pace. It would also be a good trial to see if the function-

ality can deal with the weaker consistency guarantees the queue introduces.

2.6 Incremental Migration from a Monolith to an Event-Driven Architecture: Managing Dependencies

In the last few sections, we detailed how to migrate data to new services using an event-driven approach. This section will discuss approaches when a new service might still need data from the old architecture and when the monolith has the same need for the new services.

We might find some use cases in the monolith relatively decoupled from the rest of the functionalities due to not needing many dependencies to work with. The example of the reporting module, although it needs order management information, is not a central functionality that is referenced everywhere. As mentioned in Section [2.2](#), we should use these modules to start the migration; it's often better to start with a small, easy part and build our way on top of that than trying to migrate a central functionality that is referenced throughout all modules. However, we will reach the point where we might migrate a functionality but still need data from an unrelated module inside the monolith. More than reacting to its changes, it requires the module's data to process its logic. Since the data is in the monolith, we need strategies to access that data in the service. The inverse can also be applied; other modules in the monolith might still need data that we migrated to a new service.

2.6.1 Managing a Dependency from a New Event-Driven Service to a Legacy Monolith

Using the same example in Figure [2-2](#), let's now look at the inventory module. The inventory module might need product information to process stock changes; every time someone changes the stock, the inventory module fetches product information and associates that stock change to the product. This use case differs from the reporting module since it is not reacting to the system's changes; the inventory module fetches information in the flow of its domain.

The order management calls the inventory module to remove the stock quantity the user bought. Instead of calling the inventory module directly, the order management module would publish an event signaling the user created an order. The new inventory service would react to that event but would still need the product management module information. The first most direct approach would be to request that information from the monolith, as illustrated in Figure 2-10.

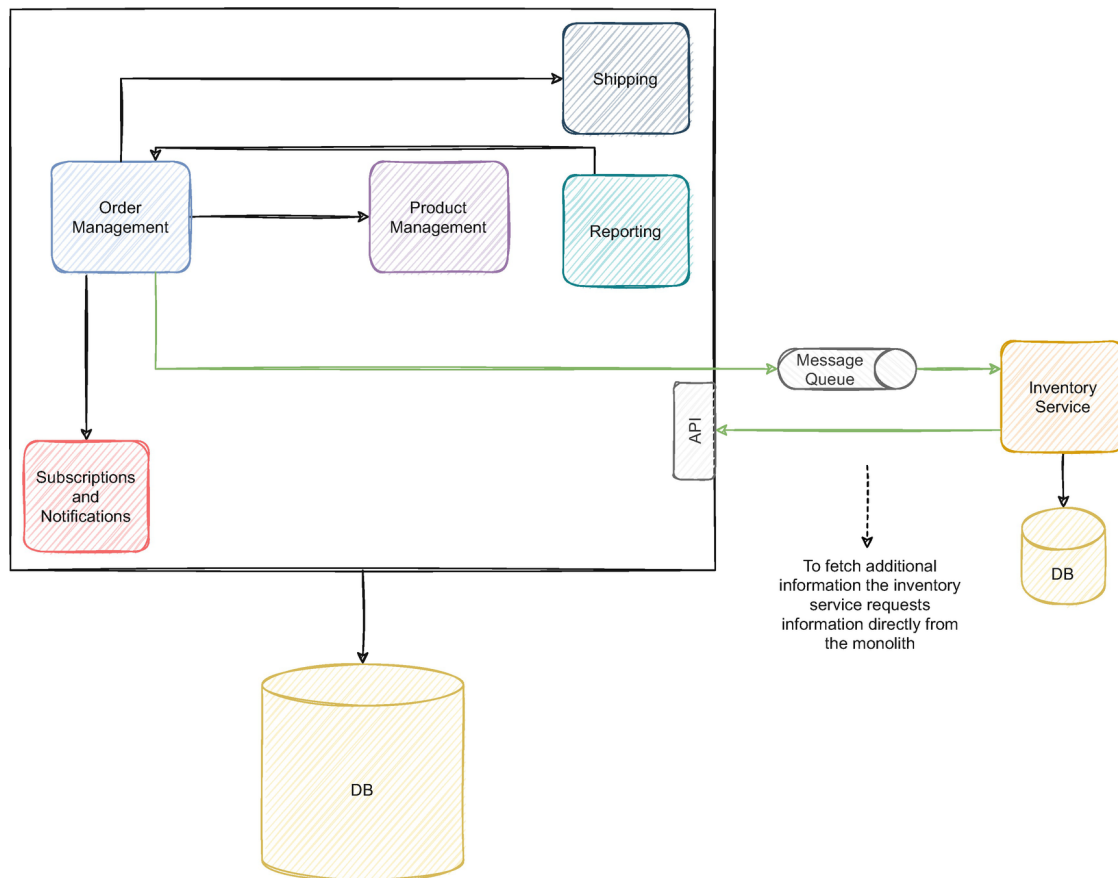


Figure 2-10 When the inventory service needs additional information that still exists on the monolith, it requests the information directly through an API

This alternative is very intuitive: we treat the dependency in the inventory service as we would call any other module inside the monolith, but instead of being inside the same application, it now makes a remote request. We often are reluctant to add functionality to the monolith, and we should be; however, adding functionality to the monolith might be a stepping stone to build the new architecture.

However, the inventory service now has a direct dependency on the monolith. If the monolith struggles with performance issues or has a catastrophic failure (like a memory leak), it will cascade to the inventory service. Scaling the inventory service will also impact the monolith; if we add more resources to the service, it will trigger more requests to the

monolith. These issues were the very thing we were trying to avoid with the decoupled nature of event-driven architectures. It is a viable choice, and we might use it sparingly to promote the migration of functionalities that need this approach. However, it must be temporary and something we should use only sparingly.

An alternative to this option is to publish the product data and use an internal view of that data inside the inventory service. This way, the inventory service remains decoupled, and we avoid the remote request to the monolith, illustrated in Figure 2-11.

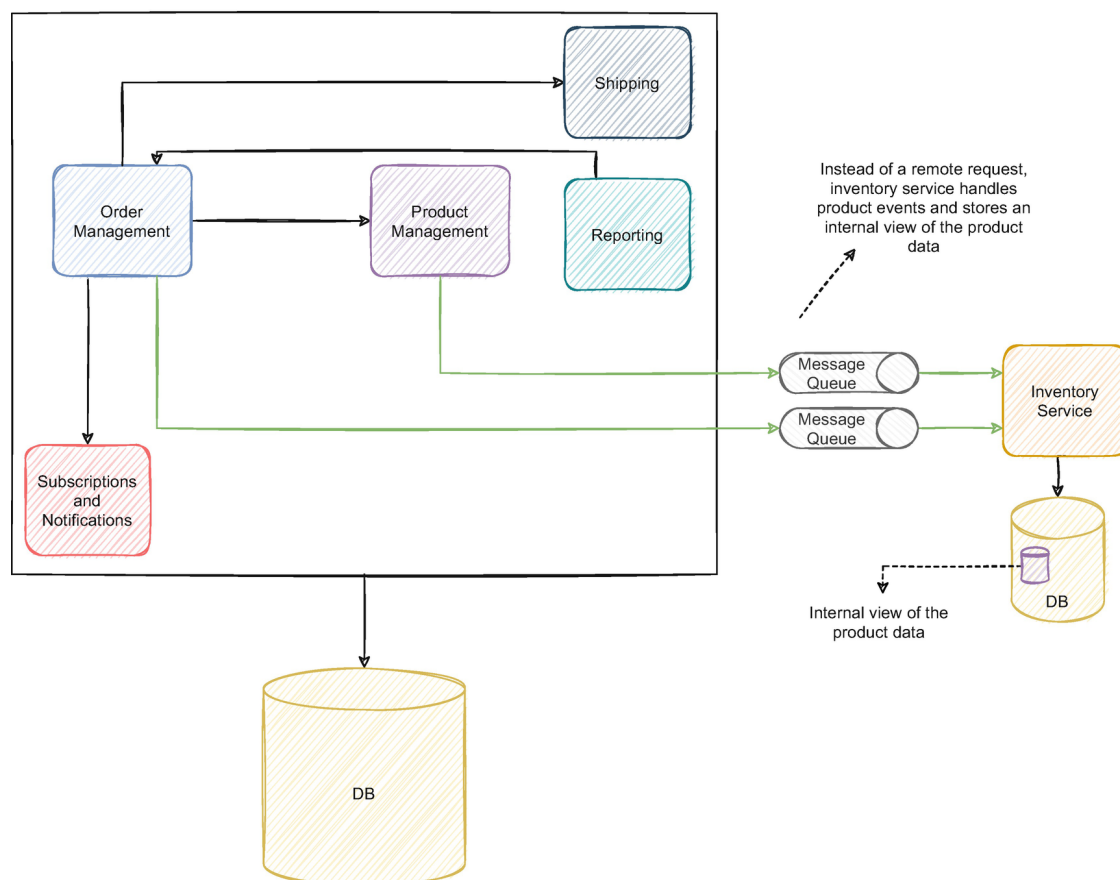


Figure 2-11 Instead of a direct remote request to the monolith, we are able to avoid a direct dependency through an internal view of the product data inside the inventory service

The inventory service, by having an internal view of the product data, would be able to process its domain logic by using that internal state instead of requesting data to the monolith. The state doesn't necessarily need to be saved inside the database; Kafka, for example (we will discuss Kafka in more detail in Chapters 3 and 6), if we use Kafka Streams, provides a way to keep that information in KTables. It is also possible to opt for other solutions to save data, discussed in Chapter 8.

We are preferring to denormalize the data in the service over having a centralized place to hold it. Besides the higher decoupling, this solution provides a performance boost since accessing local data is often faster than making a remote external request. But it also has an infrastructure and maintenance overhead of saving and maintaining that data.

2.6.2 Managing a Dependency from a Legacy Application to a New Event-Driven Service

As we discussed, by migrating modules from the monolith, we also move the data associated with those modules. There might be use cases where other modules inside the monolith require that data. A common example is UIs that display diverse information and often aggregate data from several domains.

Using the same example we used before, let's say the order management module needed to validate if the product being ordered had enough stock to fulfill the order when receiving a new order. To do the validation, the module would have to fetch the current stock information. If we move the inventory service to an independent service, the data no longer exists in the monolith.

We could follow the same approaches we discussed before. The most intuitive approach would be to request information from the new service, as illustrated in [Figure 2-12](#). When the order management module needs the stock information, it would request data from the inventory service much like it would do if the module was still inside the monolith, but instead of running the request in memory, it makes a remote request.

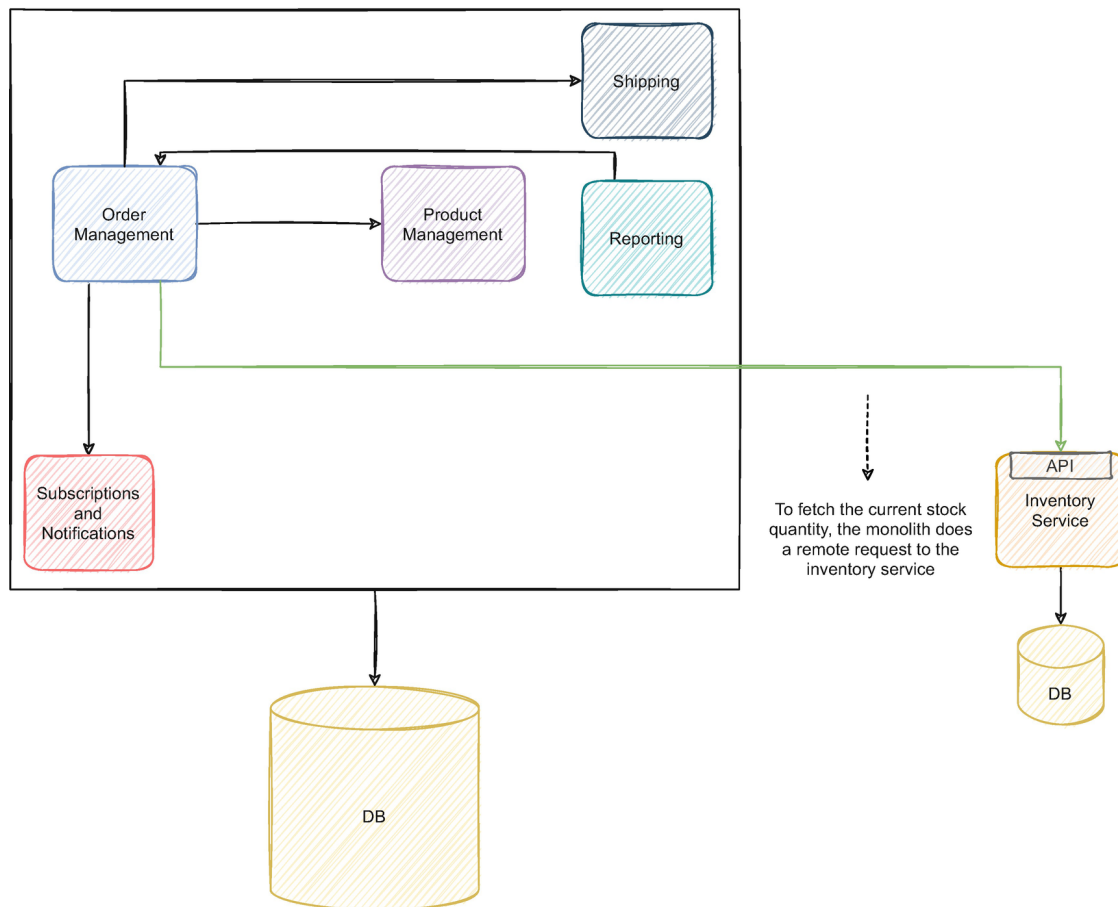


Figure 2-12 The monolith requests information from the inventory service through a remote request

Although this approach suffers from the same issues we mentioned before, they aren't as troublesome. We might still have a cascading failure if the inventory service fails, but it is less likely to happen since the service has a minimal scope. On the other hand, any change in any bounded context of the monolith has the potential to bring the whole application down. If we need to add load to the monolith, it will trigger more requests, but the inventory service is easily scalable; scaling a monolith is harder. Since this approach is straightforward and easy to implement, we might use it as a building block, and when we move the order management module, opt for a different strategy.

We could also adopt a strategy to feed the monolith's existing state with the inventory service's events. This way has the advantage of keeping both architectures decoupled, as illustrated in Figure [2-13](#).

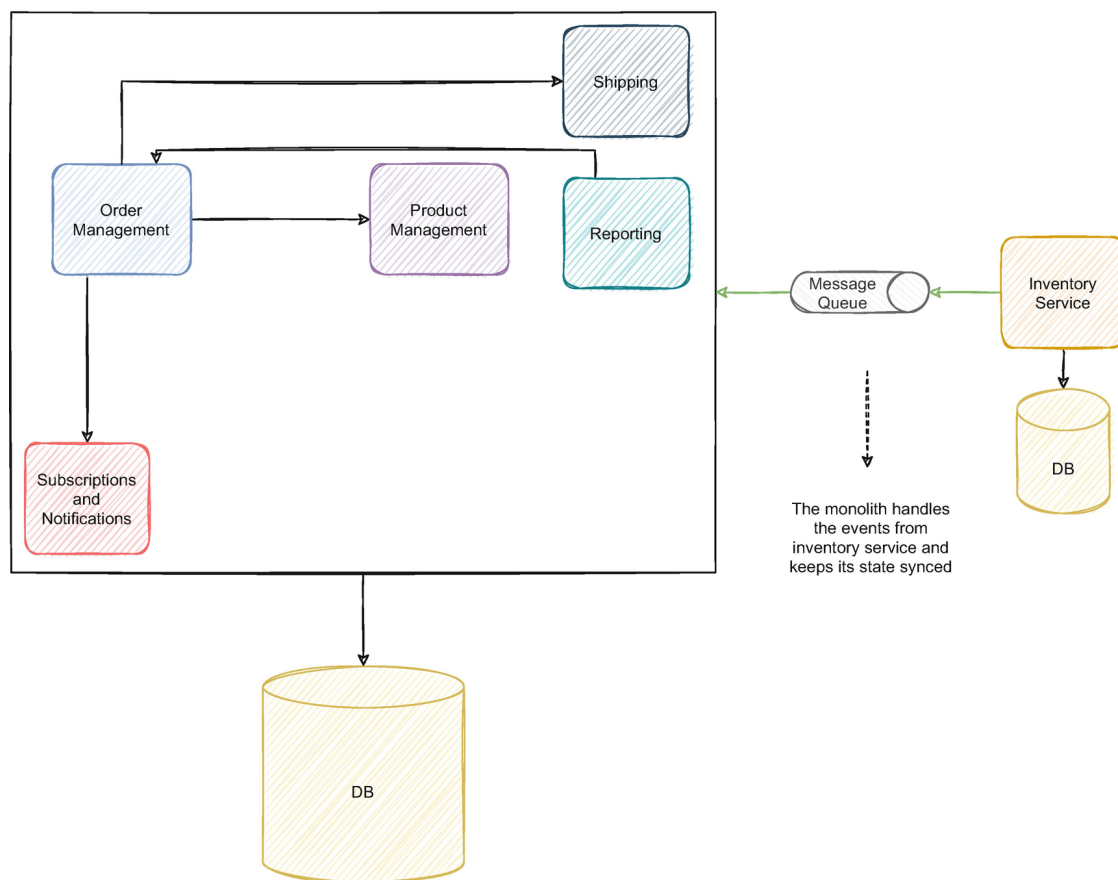


Figure 2-13 Monolith handles the events and keeps its internal view of the inventory data synced

Often monoliths have internal views that join a large amount of diverse information together. Joining information is possible since all data is stored in the same database; when deconstructing the database, maintaining this kind of functionality is challenging. A sustainable approach is to sync the information back to the monolith and feed its internal state through the new services' events. The new services are the source of truth, making the monolithic database a denormalized view of the different domains. This approach is often required since joining diverse information through HTTP requests isn't possible without requesting a lot of information and joining it in memory, a cumbersome process both performance and resource-wise.

2.7 Gradually Moving Traffic to New Microservices

In the last sections, we discussed how we could move functionality from a monolith to individual services. However, we didn't detail how to shift traffic from the monolith to the new services. This section will detail how we can gradually move that traffic without a big bang release.

Previously, we discussed that we could shift traffic to the new service once the new service is fully functional and descope the old functionality. However, we should guarantee the transition has the necessary quality guarantees. Whenever possible, we should avoid big bang releases; it is much safer to gradually deliver functionally than releasing it all at once, in a single moment.

The event-driven approaches we discussed in the previous sections allow us to build the new services gradually without impacting the current application. Exposing the data on the event queue also enables the service to asynchronously bootstrap its database. Since the message queue decouples the service and enables it to process the information asynchronously, as that process advances, it provides a way to compare the information in both databases to guarantee the data is coherent. Event-driven queues also retain the messages even after consumption; if we detect an issue that corrupted the data, we can simply read the queue from the beginning and regenerate the state. Once we are confident the data is coherent, we can start to shift requests to the new service, as illustrated in Figure [2-14](#).

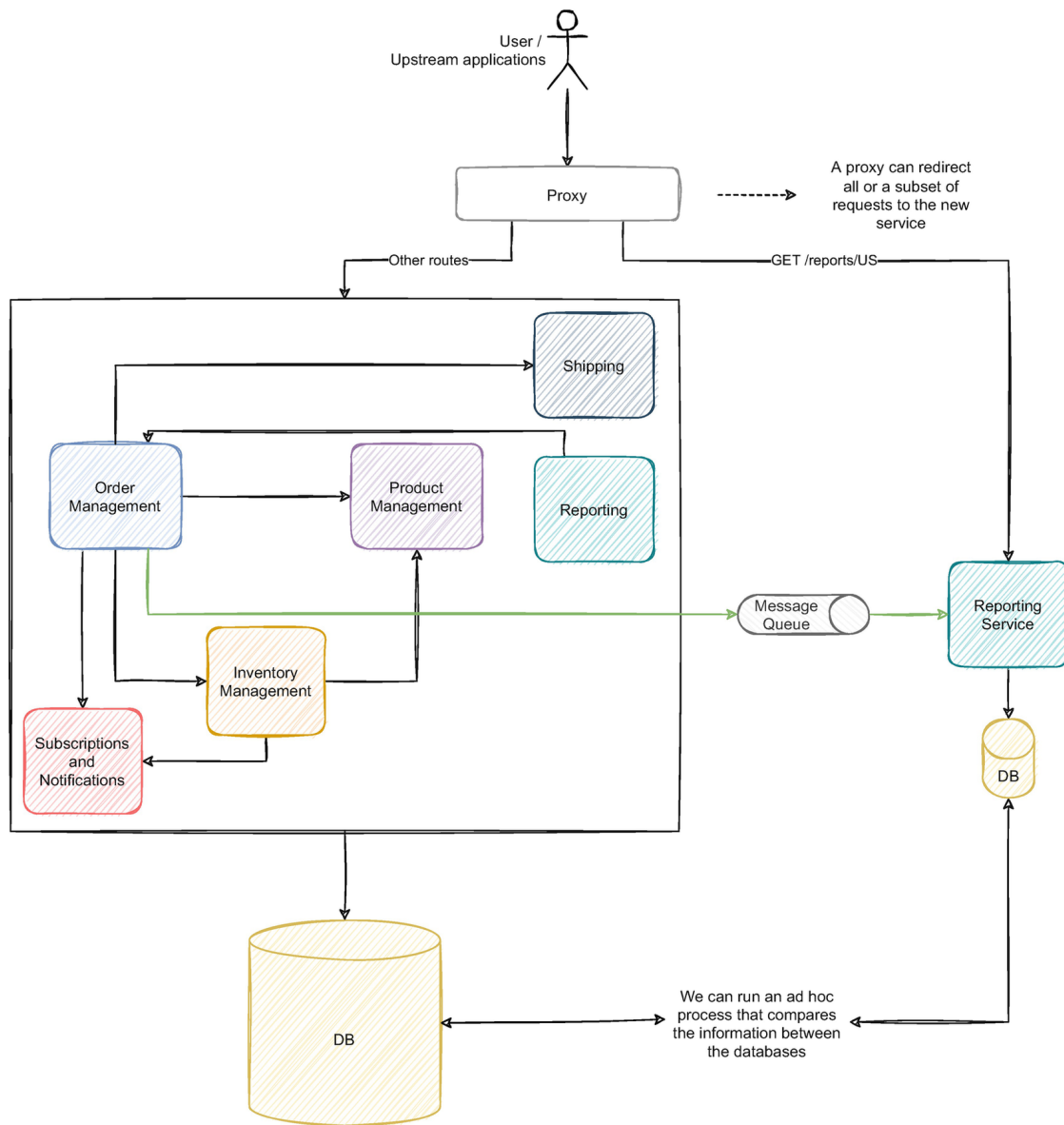


Figure 2-14 The proxy between the upstream applications and the downstream services can route the monolith or new service requests

We also don't need to route all requests at once; we can route based on the content; the proxy can filter a subset of requests to the new service and the remaining ones to the existing application, guaranteeing a gradual rollout. For example, as illustrated in Figure 2-14, the proxy can forward part of the requests based on the route. If we consult reports by country, we could forward requests for a country to the new service and keep the remaining in the existing application.

If it is impossible to set up a proxy between the upstream systems, we can use the monolith to serve as a proxy and forward the requests to the new service. We can change it to receive the request and internally call or send a message to the new service. It's not ideal since we will still be using the application we want to discontinue, but might help as a tempo-

rary solution. At the same time, the monolith’s consumers can adapt to the new services.

We can also apply content routing to messages. If the monolith is already receiving messages, like the approach in Section [2.5](#), we can forward a subset of messages to the new service and the remaining to the existing application. For example, messages with country code “US” would be consumed by the new service and ignore the others, while the monolith would do the inverse. We could also use different queues, one for the new service and another for the monolith, although we would have to duplicate the messages for both queues. This way is also possible to gradually shift message traffic between the two applications.

It is also important to maintain a rollback strategy in case things go wrong. Event-driven queues retain the messages in the queue. If we build systems that rely on those queues to process their domain logic, it’s often straightforward to roll back the system and reprocess the messages after the point the issue was introduced. Chapter [7](#) further details this rollback strategy.

2.8 Migrating from a Monolith: Two-Way Synchronization and Living with Two Sources of Truth

In Section [2.6](#), we discussed how to feed the monolith’s state after migrating the source of truth to the new services. In the previous sections, we discussed how to migrate the monolith’s functionality and data to the new architecture. However, we discussed use cases where the transition is immediate or spans during a short period. There are situations where this transition isn’t as immediate, and we might need to maintain both the old and new architecture for a considerable amount of time. We should always avoid two sources of truth; it’s complicated and hard to maintain; however, in the real world, often migrating functionality is messier than simply descopeing old functionality instantaneously.

In any migration to an event-driven microservice, our key goal is to turn the new service into the source of truth; the entry point of that do-

main's information is the new service. A single source of truth is a fundamental objective we always must aim for in any situation. However, there are times we must maintain functionalities in the old application that can't or are too hard to migrate to in a short period. Functionalities that span several domains can be hard to discontinue since each domain often belongs to different teams. Some teams may need to advance in the migration at a different time than the others. As mentioned in Section [2.7](#), we should always do this migration gradually, so even if the timing was right, synchronizing a release between several teams is more often than not a recipe for disaster. Section [2.6](#) mentions strategies to keep the monolith's state synchronized, but how does that play out when we mix strategies like those mentioned in Sections [2.3](#) and [2.6](#)? We could descope the monolith's functionality and feed the state back to the monolith in a single moment, but this is risky and such short time migrations are rare.

When we discussed CDC in Section [2.4](#), we described it as an interesting approach because there might be functionalities entangled throughout the whole application. When the monolith's domain we are trying to split has logic spread across the entire application and in stored procedures, it can be exceptionally hard to decouple it from the rest of the application. Several entry points are often dependent on other domain logic or on consumers that can't change to the new service quickly. In the company where we were trying to migrate the inventory management logic I mentioned in Section [2.4](#), we used CDC and it was going well; with CDC, we were able to capture every change and synchronize it to the new service. However, that also meant the entry points were still in legacy. Although we were able to change many of the clients to access the service directly or proxy the monolith's call to the new service, some of them were unfeasible due to the associated coupling and complexity. There were stored procedures that changed stock that joined a large amount of information from different domains like order and product. These were also business-critical functionalities; descopeing or making them unavailable for a period of time wasn't acceptable. We were able to descope them later on and move them to the new applications but only after most of the other domains left the monolith, which took years to do so; meanwhile, we had to live with two entry points and two sources of truth.

Doing so allowed the stock information to be available in the new architecture, enabling the building of new services and applications on top of that information instead of legacy applications. This while still maintaining some of the monolith's functionalities that we couldn't move.

Using the same example in Figure 2-2, let's say the user could edit the reports generated by the reporting service. The user can edit reports in the new service, but editions can also be done in the monolith due to functionality we couldn't move. Combining the approaches we discussed until now, we could publish changes from both the monolith and the reporting service to a message queue and both would listen from those queues, but only the changes that weren't from that system would be consumed.

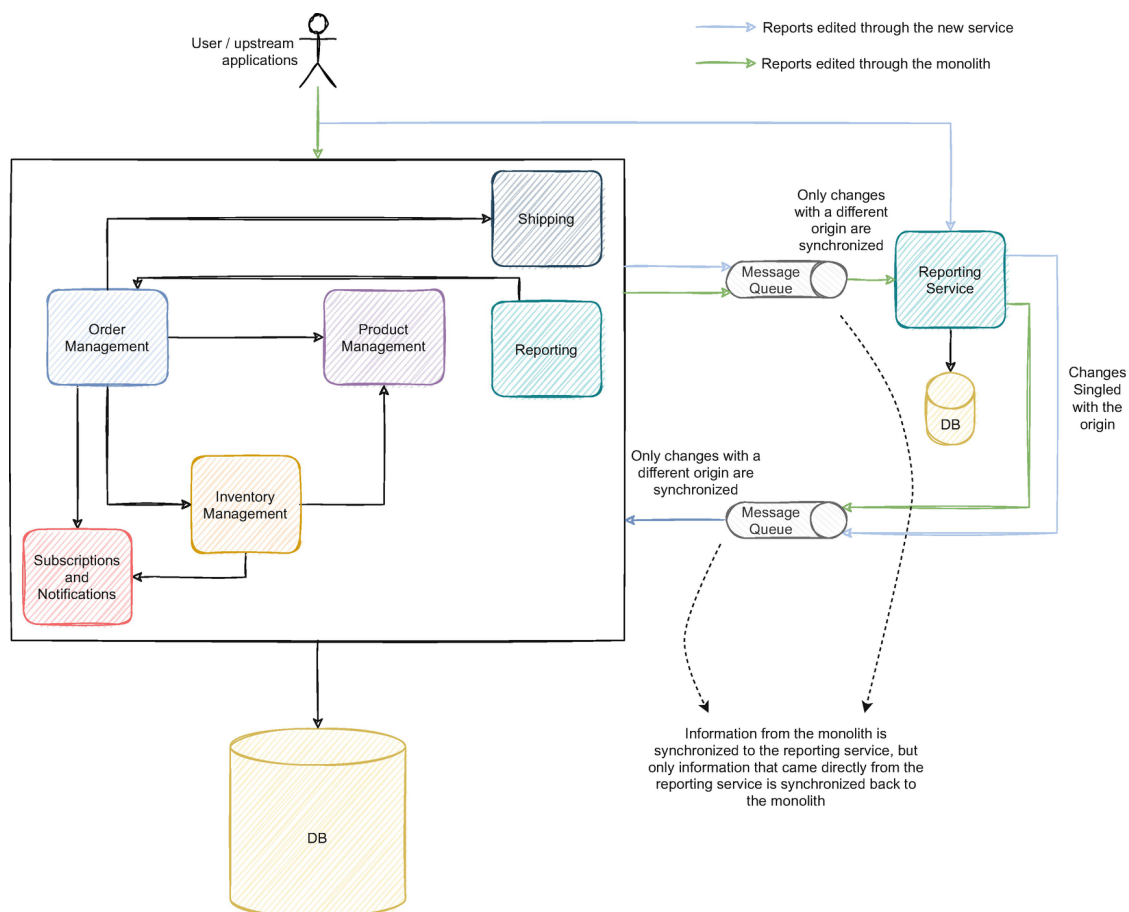


Figure 2-15 A way to maintain both systems synchronized. Both systems publish changes, and both systems process them but ignore the changes that were originated by themselves

One of the catches in this solution is breaking the cycle; we can do that, for example, with a header signaling the origin system. In Figure 2-15, if a user edits a report in the monolith, the monolith would publish the corresponding event to the message queue. The event would also need to inform the origin system of that change. The reporting service

would consume that change and publish a similar event signaling the same change to its queue. The monolith is consuming that queue but ignores the change because it originated in the monolith. Changes in the reporting service have the same flow and are published to the service's queue, consumed by the monolith, published again by the monolith, and ignored at the service.

We aim to descope the monolith's message queue; all new services should plug into the reporting service's queue. As we descope the monolith's functionalities, fewer messages should flow in the monolith's queue. Once no messages are flowing in the queue means the monolith no longer has any functionality that affects the reporting service, and we finalized the migration. Then we should also descope the monolith's queue.

There are also other details we need to consider; for example, if the user changes the same report at the same time in both systems, how would we manage the collision? We should identify events with a version or timestamp and the latest should prevail. So when consuming the messages, the systems would need to validate the message date and only update more recent changes than the system's internal state. However, these challenges aren't exclusive to two sources of truth; any horizontally scalable event-driven system can encounter parallel changes due to concurrency between instances or loss of the event's ordering. We will discuss approaches to these challenges in [Chapters 5, 6, and 7](#).

Maintaining two-way synchronization, in the long run, is extremely hard and the source of ongoing issues. Not only are incidents harder to trace since they can originate from entirely different systems, but new functionalities are also limited to the monolith's design choices. The domain logic also needs to stay coherent between the two systems. It is possible to evolve the new system without affecting the monolith due to the decoupled nature of the message queues, as long as the message's contract remains the same, but only until a point. Most of the time, we can avoid having two sources of truth, and we should do the utmost possible to prevent it. It is a solution we might keep in our toolbox, but near the bottom, other solutions are often preferable.

2.9 Summary

- An event-driven architecture has a complexity boost associated with it, and we should question whether it is the best option for our use case. We should always ask ourselves what we are trying to solve and have a clear reason to advance with the migration.
- We should always strive to do an incremental migration. Doing so will allow us to face the challenges associated with it in a sustainable manner.
- Often good candidates to be migrated first are modules with fewer dependencies and that largely benefit from the migration.
- Event-driven provides a high decoupled solution to free data from a monolith. It also provides the tools to build the new services asynchronously without impact to the monolith and the teams working with it.
- When functionalities are too coupled, too complex, or simply can't be migrated seamlessly, CDC is a valid option to access the data changes and convert them to events.
- We can use events to trigger both changes in the monolith and new services. Doing so places the source of truth in the events further promoting the mindset shift to adopt events as a single source of truth.
- When other modules are depending on the data of the functionality we are migrating, we can use events to feed that state inside those applications. A view of the state can be maintained both in the monolith and new services and be used to solve those dependencies.
- Some migrations don't need to be done fully in a single moment; we can gradually change traffic to the new applications.
- There might be use cases where a full migration can't be done in a short timespan and might take several months. It is possible to use two-way synchronization to maintain the harder functionalities in the monolith while proceeding with the migration of the remaining ones. However, it is a complex option with several impacts; we should always strive for a single source of truth.

Footnotes

¹ See this article by Kong, December 11, 2019, <https://konghq.com/press-release/2020-digital-innovation-benchmark/>

[2](#) See this article by Christian Posta, “Istio as an Example of When Not to Do Microservices,” January 8, 2020, <https://blog.christianposta.com/microservices/istio-as-an-example-of-when-not-to-do-microservices/>

[3](#) See Alexandra Noonan, “Goodbye Microservices: From 100s of problem children to 1 superstar,” July 10, 2018, <https://segment.com/blog/goodbye-microservices/>

[4](#) See “Why We Replaced Our Kafka Connector with a Kafka Consumer,” December 16, 2017, <https://hackernoon.com/why-we-replaced-our-kafka-connector-with-a-kafka-consumer-972e56bebb23>

[Support](#) [Sign Out](#)